Figure 18.13	Variable-scope-release burnup chart	329
Figure 18.14	Fixed-date-release burnup chart (with inverted product backlog)	330
Figure 19.1	When sprint planning happens	336
Figure 19.2	Sprint-planning activity	337
Figure 19.3	Two-part sprint-planning approach	339
Figure 19.4	One-part sprint-planning approach	340
Figure 19.5	Development team capacity in a sprint	341
Figure 19.6	Sprint backlog showing PBIs and task plan	345
Figure 20.1	When sprint execution happens	347
Figure 20.2	Sprint execution activity	348
Figure 20.3	Cost of multitasking	350
Figure 20.4	Mini waterfall during sprint execution—a bad idea	352
Figure 20.5	Subset of Extreme Programming technical practices	355
Figure 20.6	Example task board	356
Figure 20.7	Sprint burndown chart	358
Figure 20.8	Sprint burndown chart with trend lines	359
Figure 20.9	Sprint burnup chart	360
Figure 21.1	When the sprint review happens	363
Figure 21.2	Sprint review prework	366
Figure 21.3	Sprint review activity	369
Figure 22.1	Edward Bear illustrating the need for a retrospective	376
Figure 22.2	When the sprint retrospective happens	376
Figure 22.3	Sprint retrospective prework	378
Figure 22.4	Sprint retrospective activity	381
Figure 22.5	Aligning perspectives to create a shared context	383
Figure 22.6	Sprint event timeline	384
Figure 22.7	Emotions seismograph	385
Figure 22.8	Retrospective insight card wall	386
Figure 22.9	Insight cards clustered into similarity groups	386
Figure 22.10	Insight cards placed into predetermined groups	387
Figure 22.11	Example of dot voting	388
Figure 22.12	Sprint retrospective issues	391

Foreword
by Mike Cohn

I had lunch today at a Burger King. A sign on the wall proclaimed the restaurant the "Home of the Whopper" and then proceeded to tell me there were over a million different ways to order a Whopper. If various combinations of extra or no pickles, tomatoes, lettuce, cheese, and so on can lead to over a million ways to make a hamburger, there must be billions of possible ways to implement Scrum. And while there is no single right way, there are better and worse ways to implement Scrum.

In *Essential Scrum*, Kenny Rubin helps readers find the better ways. His isn't a prescriptive book—he doesn't say, "You must do this." Instead, he teaches the essential principles underlying success with Scrum and then gives us choices in how we live up to those principles. For example, there is no one right way for all teams to plan a sprint. What works in one company or project will fail in another. And so Kenny gives us choices. He describes an overall structure for why Scrum teams plan sprints and what must result from sprint planning, and he gives us a couple of alternative approaches that will work. But ultimately the decision belongs to each team. Fortunately for those teams, they now have this book to help them.

An unexpected benefit of *Essential Scrum* is the visual language Kenny introduces for communicating about Scrum. I found these images very helpful in following along with the text, and I suspect they will become commonplace in future discussions of Scrum.

The world has needed this book for a long time. Scrum started as a small concept. The first book to talk about it—*Wicked Problems, Righteous Solutions* in 1990 by DeGrace and Stahl—did so in six pages. But in the more than 20 years since that book appeared, Scrum has expanded. New roles, meetings, and artifacts have been introduced and refined. With each new piece that was added, we were at risk of losing the heart of Scrum, that part of it that is about a team planning how to do something, doing some small part of it, and then reflecting on what the team members did and how well they did it together.

With *Essential Scrum*, Kenny brings us back to the heart of Scrum. And from there teams can begin to make the decisions necessary to implement Scrum, making it their own. This book serves as an indispensable guide, helping teams choose among the billions of possible ways of implementing Scrum and finding one that leads to success.

—Mike Cohn
Author of *Succeeding with Agile, Agile Estimating and Planning,* and *User Stories Applied*
www.mountaingoatsoftware.com

This page intentionally left blank

Foreword
by Ron Jeffries

When Kenny asked me to write a foreword for *Essential Scrum,* I was thinking, "This will be quick and easy; it must be a short book going straight to a simple description of what Scrum is." I knew Kenny's work, so I knew it would be a good read, and short, too. What could be better!

Imagine my surprise and delight when I found that this book covers just about everything you'll need to know about Scrum, on the first day or years into your use of Scrum. And Kenny doesn't stop there. He starts with the central ideas, including the agile principles that underlie all the agile methods, and a quick view of the Scrum framework. Then he drills in, deeper and deeper. It's still a good read, and it's quite comprehensive as well.

Kenny covers planning in good detail, looking at requirements, stories, the backlog, estimation, velocity. Then he takes us deeper into the principles and helps us deal with all the levels of planning and all the time horizons. He describes how sprints are planned, executed, reviewed, and improved. And throughout, he gives us more than the basics, highlighting key issues that you may encounter as you go along.

My own focus in Scrum and agile is on the necessary developer skills to ensure that teams can deliver real, running, business-focused software, sprint after sprint. Kenny helps us understand how to use ideas like velocity and technical debt safely and well. Both of these are critical topics, and I commend them to your attention.

Velocity tells us how much the team is delivering over time. We can use it to get a sense of how much we're getting done and whether we're improving. Kenny warns us, however, that using velocity as a performance measure is damaging to our business results, and he helps us understand why.

Technical debt has become a very broad term, referring to almost everything that could go wrong in the code. Kenny helps us tease apart all the various meanings and helps us understand why we care about these seemingly technical details. In particular, I like his description of how putting a team under pressure will inevitably damage our prospects of getting a good product on time.

Scrum, like all agile methods, relies on an exploratory approach with rapid feedback. Kenny tells a story of his brief use of punch cards, and it reminded me of my earliest experience with computing, many years before Kenny saw his first punch card.

As a college student, I was lucky enough to get a job as a sort of intern at Strategic Air Command headquarters in Omaha. In those days all computing was on cards. My

cards got sent down several floors underground at SAC HQ and run on the computer that would run the war, if we ever had one. I was lucky to get one or two runs a day.

As soon as my security clearance came through, I would go down to the computer room in the middle of the night. I would sweet-talk Sergeant Whittaker into letting me run my own programs, sitting at the console of the machine—yes, the machine whose main job was to launch a nuclear attack. Rest easy, though: The red button was not in that room.

Working hands-on with the machine, I got ten times as much work done as when I had to wait for my cards to be taken down and my listings to be brought back up. Feedback came faster, I learned faster, and my programs worked sooner.

That's what Scrum is about. Instead of waiting months or even years to find out what the programmers are doing, in Scrum we find out every couple of weeks. A Scrum product owner with a really good team will be seeing actual features taking shape every few days!

And that is what Kenny's book is about. If you're new to Scrum, read it through from beginning to end. Then keep it nearby. If you've been doing Scrum for a while, scan it, then keep it nearby.

When you find yourself thinking about something that's happening to your team, or wondering about different things to try, pick up this book and look around. Chances are you'll find something of value.

—Ron Jeffries

PREFACE

This book discusses Essential Scrum—the things you have to know if you're going to be successful when using Scrum to develop innovative products and services.

What Is Essential Scrum?

Scrum is based on a small set of core **values**, **principles**, and **practices** (collectively the **Scrum framework**). Organizations using Scrum should embrace the Scrum framework in its entirety, perhaps not through the entire organization all at once, but certainly within the initial teams that will use Scrum. Embracing all of Scrum does not mean, however, that organizations must implement Scrum according to some cookie-cutter, one-size-fits-all formula. Rather, it means that organizations should always stay true to the Scrum framework while choosing an appropriate blend of **approaches** for their Scrum implementations.

Essential Scrum combines the values, principles, and practices of Scrum with a set of tried-and-true approaches that are consistent with, but not mandated by, the Scrum framework. Some of these approaches will be appropriate to your situation; others will not. Any approach will need to be inspected and adapted to your unique circumstances.

Origins of This Book

As an agile/Scrum coach and trainer, I am frequently asked for a reference book for Scrum—one that provides a comprehensive overview of the Scrum framework and also presents the most popular approaches for applying Scrum. Because I have been unable to find a single book that covers these topics at a level deep enough to be useful to today's practitioners, I found myself recommending a collection of books: a few that discuss the Scrum framework but are out of date or incomplete; several highly regarded agile books that do not focus solely on Scrum; and a handful that are focused on a specific aspect of Scrum or a specific approach but do not cover the full Scrum framework in depth. That's a lot of books for someone who just wants a single, stand-alone resource that covers the essentials of Scrum!

The originators of Scrum (Jeff Sutherland and Ken Schwaber) do have a Scrum-specific publication called "The Scrum Guide." This short document (about 15 pages) is described by its authors as the "definitive rule book of Scrum and the

documentation of Scrum itself" (Schwaber and Sutherland 2011). They equate their document to the rules of the game of chess, "describing how the pieces move, how turns are taken, what is a win, and so on." Although useful as a Scrum overview or rule book, "The Scrum Guide" is by design not intended to be a comprehensive source of essential Scrum knowledge. Extending the authors' analogy, giving a new Scrum team just "The Scrum Guide" and expecting good results would be like giving a new chess player a 15-page description of the rules of chess and expecting her to be able to play a reasonable game of chess after reading it. It just isn't a stand-alone resource.

This book, *Essential Scrum,* is an attempt to be the missing single source for essential Scrum knowledge. It includes an in-depth discussion of Scrum's principles, values, and practices—one that in most cases agrees with other agile thought leaders and "The Scrum Guide." (Where this book offers a different perspective from what is widely promoted elsewhere, I point it out and explain why.) This book also describes approaches that are consistent with the Scrum framework and that have been used successfully by me and teams I have coached. I did not intend for this book to replace other books that provide a deep vertical treatment of a given Scrum practice or approach. Such books are complementary to and extend this book. Rather, think of *Essential Scrum* as the starting point on the journey of using Scrum to delight customers.

Intended Audience

For the many thousands of people who have taken my Working on a Scrum Team, Certified ScrumMaster, and Certified Scrum Product Owner classes, and the many teams I have coached, this book will refresh and perhaps even clarify topics we have already discussed. And for the even larger number of people with whom I have not yet had the pleasure of working, this book will either be your first introduction to Scrum and agile or it will be a chance to look at Scrum in a different light and perhaps even improve how you perform Scrum.

I did not write this book for any one specific role—this is not a book specifically for product owners, or ScrumMasters, or members of the development team. Instead, it is a book intended to give everyone involved with Scrum, from all the members of the Scrum team to those with whom they interact in the organization, a common understanding of Scrum based on a core set of concepts with a clear vocabulary for discussing them. With this shared foundation my hope is that your organization will be in a better position to successfully use Scrum to deliver business value.

I imagine that every Scrum team member would have a copy of this book on her desk open to a chapter relevant to the work at hand. I also envision managers at all levels of the organization reading it to understand why Scrum can be an effective approach for managing work and to understand the type of organizational change that may be necessary to successfully implement Scrum. Organizations using or

planning to use an agile approach other than Scrum will also find the information relevant and helpful to their specific agile adoption.

Organization of This Book

This book begins with a brief introduction to Scrum (Chapter 1) and concludes with a discussion of where you might go next (Chapter 23). The remaining chapters are organized into four parts:

- Part I—Core Concepts (Chapters 2–8): Scrum framework, agile principles, sprints, requirements and user stories, product backlog, estimating and velocity, and technical debt
- Part II—Roles (Chapters 9–13): product owner, ScrumMaster, development team, Scrum team structures, and managers
- Part III—Planning (Chapters 14–18): Scrum planning principles, multilevel planning, portfolio planning, envisioning/product planning, and release planning
- Part IV—Sprinting (Chapters 19–22): sprint planning, sprint execution, sprint review, and sprint retrospective

How to Use This Book

As you would expect, I wrote the book assuming that most people would read it linearly from front to back. If you are new or newer to Scrum, you should take this approach because the chapters do tend to build on one another. That being said, if you are looking for one place to get an end-to-end overview of the Scrum framework (a highly visual Scrum primer), read and reference Chapter 2.

For those who are more familiar with Scrum, you can use this book as a Scrum reference guide. If you're interested in sprint retrospectives, jump directly to Chapter 22. If you are interested in exploring the nuances of the product backlog, jump directly to Chapter 6. I highly recommend, however, that everyone, even those familiar with Scrum, read Chapter 3 in its entirety. The principles laid out there form the foundation of the Scrum framework and the rest of the book. It is not simply a restatement of the values and principles of the Agile Manifesto (Beck et al. 2001) that is common in many other written descriptions of Scrum.

Visual Icon Language

I am proud to include in this book a new visual language for describing Scrum. This language is composed from a vocabulary of icons that have been designed to capture essential Scrum roles, artifacts, and activities. This visual Scrum language is an

effective way to communicate concepts and improves the overall shared understandability of Scrum. If you are interested in obtaining and using the new full-color visual Scrum language art (this book is printed in two colors), visit www.innolution.com for details. This website will also host a variety of resources and discussions related to the book.

Let's Get Started

So, whatever your role, whatever your situation, you have picked up this book for a reason. Spend a little time getting to know Scrum. In the pages that follow you just might find a powerful framework that you can make your own, allowing you to substantially improve the way you develop and deliver products and services to delight your customers.

Acknowledgments

This book would not have been possible without the input of many people, including the thousands of people who have attended my agile-related classes and coaching sessions. By mentioning some people by name, I run the risk of failing to mention others. To those whose names I fail to mention, please know that all of our discussions and email exchanges have been invaluable to me and have definitely influenced this book!

There are three people in particular I would like to thank: Mike Cohn, Rebecca Traeger, and Jeff Schaich. Without the unique involvement of each, this book would be a mere shadow of itself.

Mike Cohn has been a friend and colleague since we first worked together at Genomica in 2000. He was gracious enough to include my book in the Mike Cohn Signature Series; by being affiliated with Mike and the other prestigious authors in that book series, "I look good by the company that I keep," as my parents would say. Mike was my go-to person whenever I wanted to bounce around ideas or discuss book strategies. He always made time in his insane schedule to review each chapter and give me his thoughtful feedback. Working with Mike all these years has been a very rewarding experience—one that I hope will continue long into the future.

Rebecca Traeger has been my personal editor on this book. We have worked together since my days as managing director of the Scrum Alliance in 2007. At that time Rebecca was the editor of the Scrum Alliance website and through that work (and much more since) became the industry's foremost editor on agile-related materials. Early on in writing this book I reached out to Rebecca and asked if she would work with me again, and to my good fortune, she agreed. Nobody saw any chapter unless Rebecca had seen it first. At times her feedback would make me blush, because she frequently improved how I said something, making it sound both more understandable and approachable. If you just love a section of this book, you can be sure Rebecca had her hands in it. If you don't, I probably foolishly chose to ignore her recommendations.

Jeff Schaich is an artist/designer extraordinaire. We have worked on so many different art projects that I can't recall them all. Early on in the formulation of this book I wanted to create an agile/Scrum icon vocabulary to use as the basis for my training presentations and many of the over 200 figures in the book. I knew that I needed a great designer to pull off this feat. Jeff agreed to take on the challenge. There are times when this book seemed like two different projects—writing the content and creating

the artistic concepts. I'm honestly not sure which took more time. I am sure, however, that without Jeff's artistic input, this book would have suffered immeasurably.

I am deeply honored to have both Mike Cohn and Ron Jeffries, two luminaries in the agile community, write forewords for the book! In their own unique ways each has done a great job of properly placing the book in context and opening the door for a discussion of Essential Scrum. Also, Mike, stop eating at Burger King, and Ron, thanks for not pushing the red button!

I'd also like to thank the many people who took time out of their busy schedules to review chapters and send me their feedback. Let me start by mentioning reviewers who provided extensive feedback: Joe Balistrieri, Johannes Brodwall, Leyna Cotran, Martine Devos, Scott Duncan, Ilan Goldstein, John Hebley, Geir Hedemark, James Kovacs, Lauri Mackinnon, Robert Maksimchuk, and Kevin Tureski.

In addition, I would like to thank other reviewers who provided excellent feedback on select chapters: Lyssa Adkins, John Bauer, Sameer Bendre, Susan Briscoe, Pawel Brodzinski, Rowan Bunning, Josh Chappell, Lisa Crispin, Ward Cunningham, Cornelius Engelbrecht, Julia Frazier, Brindusa Gabur, Caroline Gordon, Drew Jemilo, Mike Klimkosky, Tom Langerhorst, Bjarne Larsen, Dean Leffingwell, Maurice le Rutte, David Luzquiños, Lv Yi, Shay McAulay, Armond Mehrabian, Sheriff Mohamed, Cuan Mulligan, Greg Pease, Roman Pichler, Jacopo Romei, Jens Schauder, Bill Schroeder, Yves Stalgies, Branko Stojaković, Howard Sublett, Julie Sylvain, Kevin Tambascio, Stephen Wolfram, and Michael Wollin.

I would also like to thank the staff at Pearson who were great partners in this project. They tolerated my delays with patience and always offered encouragement. Special thanks to Chris Guzikowski, who oversaw the whole thing from soup to nuts. He was there from my first Pearson meeting at a pub in Lexington, MA, through the final production. I would also like to thank Olivia Basegio for adeptly handling logistics and Julie Nahil who did a fantastic job overseeing the project. In addition, thanks to Barbara Wood for the great job of helping polish the manuscript and Gail Cocker for pulling all of the art together into a coherent and beautiful whole.

I am also grateful to my assistant, Lindsey Kalicki, to whom I was able to offload many important tasks so that I could stay focused on book development. I am lucky to be able to work with such a skilled professional.

Most of all, I would like to acknowledge my family—Jenine, Jonah, and Asher—and the critical role that they played. I have asked so very much from them during the long effort of creating this book. No amount of gratitude can make up for the family pressure it caused and our lost time together.

Jenine is my loving soulmate and has stuck by me through all of the ups and downs of writing this book. The sacrifices she made so that I could write would double the size of this book if I tried to list them all. I couldn't have done it without her!

Funny thing is, a year after we were married in 1993, I published my first book, *Succeeding with Objects*. At that time Jenine made me promise that I would never write another book again. Luckily for me, after 15 years memories fade and the

crushing workload doesn't seem as bad in hindsight, so when she urged me to write this one I was surprised to say the least! She hasn't yet told me I can't do book number three, but I suspect it might be 15 more years before the memory of this one fades enough for either of us to want me to write another one!

I also deeply appreciate the loving support from my sons, Jonah and Asher. They gave up time with their dad so that I could write. They were always there to bounce around ideas and to give input on the book. A number of their content and art suggestions have made their way into the book—and it's better because of them! I hope they learned the value of perseverance and that even the most daunting work can be completed if you take it a step at a time and don't give up.

Finally, I would like to acknowledge my mom, Joyce Rubin (Genesha Esther bat Avrahm), for all of the love and support she gave me. Without her influence this book would never have been possible. Sadly, she did not survive to see its publication. Her passing in January 2012 left a void in my life and the lives of her family that can never be filled. She was a very special person to the many whose lives she touched. Mom, I miss you more than I can possibly express.

This page intentionally left blank

About the Author

Kenny Rubin provides Scrum and agile training and coaching to help companies develop products in an effective and economically sensible way. A Certified Scrum Trainer, Kenny has trained over 18,000 people on agile and Scrum, Smalltalk development, managing object-oriented projects, and transition management. He has coached over 200 companies, ranging from start-ups to Fortune 10.

Kenny was the first Managing Director of the worldwide Scrum Alliance, a nonprofit organization focused on the successful adoption of Scrum. In addition to this book, Kenny is also the coauthor of the 1995 book *Succeeding with Objects: Decision Frameworks for Project Management*. He received his B.S. in Information and Computer Science from the Georgia Institute of Technology and his M.S. in Computer Science from Stanford University.

Kenny's background is rooted in the object-oriented technology community. He started as a Smalltalk developer on a NASA-funded project back in 1985 and developed the first blackboard expert system outside of LISP. In 1988 he was fortunate to join ParcPlace Systems, a start-up company formed as a Xerox PARC spin-off, whose charter was to bring object-oriented technology out of the research labs and release it to the world. As a Smalltalk development consultant with many different organizations in the late 1980s and throughout the 1990s, Kenny was an early adopter of agile practices. His first use of Scrum was in 2000 for developing bioinformatics software.

In the course of his career, Kenny has held many roles, including successful stints as a Scrum product owner, ScrumMaster, and member of development teams. In addition, he has held numerous executive management roles: CEO, COO, VP of Engineering, VP of Product Management, and VP of Professional Services. He has also overseen the development of five commercial software product suites, generating over $200M in aggregate revenue. In addition, he has been directly involved in raising over $150M in venture capital funding and assisted in taking two companies public on the NASDAQ.

His multifaceted background gives Kenny the ability to understand (and explain) Scrum and its implications equally well from multiple perspectives: from the development team to the executive board.

This page intentionally left blank

Chapter 1
INTRODUCTION

On June 21, 2000, I was employed as Executive Vice President at Genomica, a bio-informatics company in Boulder, Colorado. I remember the date because my son Asher was born at one o'clock that morning.

His birth was a good start to the day. Asher was actually born on his predicted due date (in the United States this happens about 5% of the time). So we (really my wife, Jenine) had finished our nine-month "project" on schedule. And to top things off, Asher had a very high Apgar score, indicating that we had produced a healthy, good-quality result! Our biggest stakeholder, our older son, Jonah, was thrilled to have a younger brother. On time, high quality, and delighted stakeholders—it truly was a good day!

After a brief nap, I checked email and saw that the CEO of Genomica had sent an urgent message asking me to be at a board of directors' meeting at 8:00 a.m. that same day. Begrudgingly, I left the hospital and went to the meeting.

When I arrived, I was told that the VP of Engineering had been fired the night before and I had now inherited the 90-person engineering team. I wasn't surprised. For several months the executive team and board had been discussing Genomica's inability to deliver valuable products on time and with acceptable quality, and the VP of Engineering was at the center of that discussion.

It was now my responsibility to oversee the effort of substantially improving the results of our product development organization. I remember being struck by the irony of that day's successful delivery and my new responsibilities.

Because I was already quite busy overseeing sales and marketing, I was told that at my discretion I could hire a new VP of Engineering to report to me. The person I chose to hire was Mike Cohn (Cohn 2004; Cohn 2006; Cohn 2010), and Scrum was the approach that we decided to use.

What Is Scrum?

Scrum is an **agile** approach for developing innovative products and services. Figure 1.1 shows a simple, generic, agile development approach.

With an agile approach, you begin by creating a **product backlog**—a prioritized list of the features and other capabilities needed to develop a successful product. Guided by the product backlog, you always work on the most important or highest-priority items first. When you run out of resources (such as time), any work that didn't get completed will be of lower priority than the completed work.

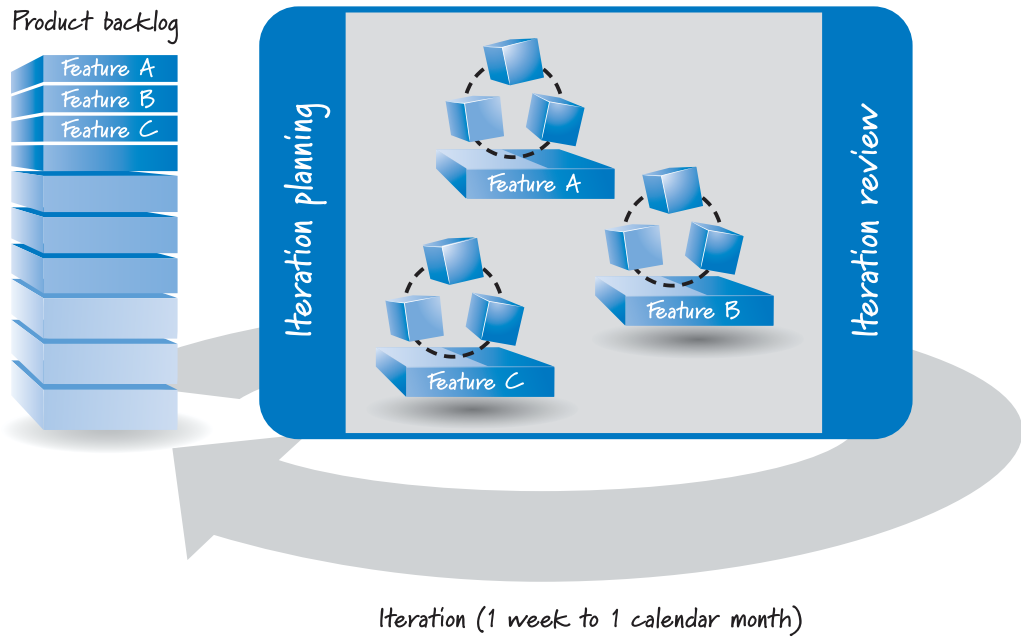


FIGURE 1.1 Agile development overview

The work itself is performed in short, timeboxed **iterations**, which usually range from a week to a calendar month in length. During each iteration, a self-organizing, **cross-functional team** does all of the work—such as designing, building, and testing—required to produce completed, working features that could be put into production.

Typically the amount of work in the product backlog is much greater than can be completed by a team in one short-duration iteration. So, at the start of each iteration, the team plans which high-priority subset of the product backlog to create in the upcoming iteration. In Figure 1.1, for example, the team has agreed that it can create features A, B, and C.

At the end of the iteration, the team reviews the completed features with the stakeholders to get their feedback. Based on the feedback, the product owner and team can alter both what they plan to work on next and how the team plans to do the work. For example, if the stakeholders see a completed feature and then realize that another feature that they never considered must also be included in the product, the product owner can simply create a new item representing that feature and insert it into the product backlog in the correct order to be worked on in a future iteration.

At the end of each iteration, the team should have a potentially shippable product (or increment of the product), one that can be released if appropriate. If releasing after each iteration isn't appropriate, a set of features from multiple iterations can be released together.

As each iteration ends, the whole process is begun anew with the planning of the next iteration.

Scrum Origins

Scrum's rich history can be traced back to a 1986 *Harvard Business Review* article, "The New New Product Development Game" (Takeuchi and Nonaka 1986). This article describes how companies such as Honda, Canon, and Fuji-Xerox produced world-class results using a scalable, team-based approach to **all-at-once product development**. It also emphasizes the importance of empowered, self-organizing teams and outlines management's role in the development process.

The 1986 article was influential in weaving together many of the concepts that gave rise to what today we call Scrum. Scrum is not an acronym, but rather a term borrowed from the sport of rugby, where it refers to a way of restarting a game after an accidental infringement or when the ball has gone out of play. Even if you are not a rugby aficionado, you have probably seen a scrum where the two sets of forwards mass together around the ball with locked arms and, with their heads down, struggle to gain possession of the ball.

Takeuchi and Nonaka used the metaphors of rugby and the scrum to describe product development:

> *The . . . "relay race" approach to product development . . . may conflict with the goals of maximum speed and flexibility. Instead a holistic or "rugby" approach—where a team tries to go the distance as a unit, passing the ball back and forth—may better serve today's competitive requirements.*

In 1993, Jeff Sutherland and his team at Easel Corporation created the Scrum process for use on a software development effort by combining concepts from the 1986 article with concepts from object-oriented development, empirical process control, iterative and incremental development, software process and productivity research, and complex adaptive systems. In 1995, Ken Schwaber published the first paper on Scrum at OOPSLA 1995 (Schwaber 1995). Since then, Schwaber and Sutherland, together and separately, have produced several Scrum-specific publications, including *Agile Software Development with Scrum* (Schwaber and Beedle 2001), *Agile Project Management with Scrum* (Schwaber 2004), and "The Scrum Guide" (Schwaber and Sutherland 2011).

Though Scrum is most commonly used to develop software products, the core values and principles of Scrum can and are being used to develop different types of products or to organize the **flow** of various types of work. For example, I have worked with organizations that have successfully used Scrum for organizing and managing the work associated with hardware development, marketing programs, and sales initiatives.

Why Scrum?

So what made an agile approach like Scrum a good choice for Genomica? First, it was clear that Genomica's previous approach to development simply wasn't working. That was the bad news; the good news was that most everyone agreed.

Genomica operated in a complex domain where more was unknown than known. We built products that had never been built before. Our focus was on bleeding-edge, continuously evolving, state-of-the-art, discovery informatics platforms that research scientists would use to help discover the next blockbuster molecule. We needed a way of developing that would allow us to quickly explore new ideas and approaches and learn fast which solutions were viable and which were not. We had a strategic corporate partner to whom we needed to show working results every few weeks or so to get feedback, because our product had to integrate with its core line of DNA sequencers. This need for rapid exploration and feedback did not mesh well with the detailed, up-front planning we had been doing.

We also wanted to avoid big up-front architecture design. A previous attempt to create a next generation of Genomica's core product had seen the organization spend almost one year doing architecture-only work to create a grand, unified bioinformatics platform. When the first real scientist-facing application was put on top of that architecture, and we finally validated design decisions made many months earlier, it took 42 seconds to tab from one field on the screen to the next field. If you think a typical user is impatient, imagine a molecular biologist with a Ph.D. having to wait 42 seconds! It was a disaster. We needed a different, more balanced approach to design, which included some design up front combined with a healthy dose of emergent, just-in-time design.

We also wanted our teams to be more cross-functional. Historically Genomica operated like most organizations. Development would hand off work to the test teams only after it was fully completed. We now had a desire for all team members to synchronize frequently—daily was the goal. In the past, errors were compounded because important issues were being discussed too late in the development effort. People in different areas weren't communicating frequently enough.

For these reasons, and others, we determined that Scrum would be a good fit for Genomica.

Genomica Results

When we chose to embrace Scrum, it was not well known; the first Scrum book didn't appear until the following year (Schwaber and Beedle 2001). However, we pulled together the available information and did the best we could, which was substantially better than we had done before (see Table 1.1).

From an effort perspective, with Scrum development we required one-tenth the amount of effort (calculated in person-months) compared to our previous use of a

TABLE 1.1 Genomica Scrum Results

Measure	Waterfall	Scrum
Effort	10x	1x
Velocity	1x	7x
Customer satisfaction	Poor	Excellent

plan-driven, **waterfall-style** approach to develop a comparable amount of product functionality. Equally important, the Scrum development progressed at seven times the velocity of the waterfall development, meaning that per unit of time, the Scrum development produced about seven times more valuable features than the waterfall development. Even more compelling was that we delivered the software to our partner in a time frame that met the expectations for the launch of its new hardware platform. This enabled us to reinforce a long-term partnership that substantially increased the shareholder value of Genomica.

Can Scrum Help You?

The Genomica pre-Scrum experience of building features that nobody wanted and delivering those features late and with poor quality is not uncommon. Genomica, like many other organizations, had survived by being no worse than its competitors. I saw the same problems when I first started working in commercial software development in the mid-1980s. And for many, after nearly 30 years, the situation hasn't improved.

Today, if you gathered together your business people and developers and asked them, "Are you happy with the results of our software development efforts?" or "Do you think we deliver good customer value in a timely, economical, and quality manner?" what would they say?

More often than not, the people I meet during my worldwide training and coaching answer both questions with a resounding "No." This is followed by a chorus of "Project failure rate is unacceptably high"; "Deliverables are late"; "Return on investment frequently falls short of expectations"; "Software quality is poor"; "Productivity is embarrassing"; "No one is accountable for outcomes"; "Employee morale is low"; "Employee turnover is too high." Then there's the under-the-breath snicker that accompanies the tongue-in-cheek "There must be a better way."

Yet even with all this discontent, most people seem resigned to the fact that dissatisfaction is just part of the reality of software development. It doesn't have to be.

Organizations that have diligently applied Scrum are experiencing a different reality (see Figure 1.2).

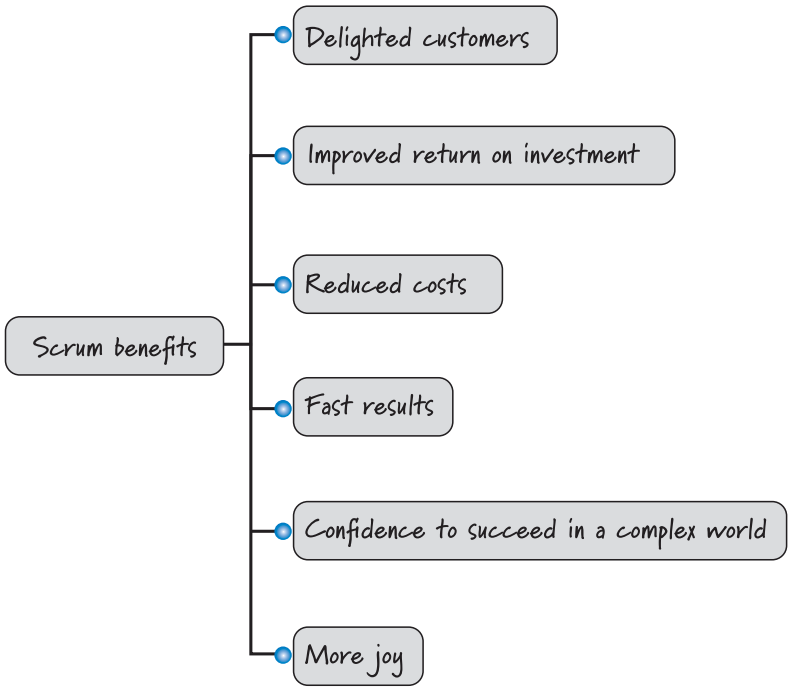


FIGURE 1.2 Scrum benefits

These organizations are repeatedly delighting their customers by giving them what they really want, not just the features they might have specified on the first day when they knew the least about their true needs. They are also seeing an improved return on investment by delivering smaller, more frequent **releases**. And, by relentlessly exposing organizational dysfunction and **waste**, these organizations are able to reduce costs.

Scrum's focus on delivering working, integrated, tested, business-valuable features each iteration leads to results being delivered fast. Scrum is also well suited to help organizations succeed in a complex world where they must quickly adapt based on the interconnected actions of competitors, customers, users, regulatory bodies, and other stakeholders. And Scrum provides more joy for all participants. Not only are customers delighted, but also the people doing the work actually enjoy it! They enjoy frequent and meaningful collaboration, leading to improved interpersonal relationships and greater mutual trust among team members.

Don't get me wrong. Though Scrum is an excellent solution for many situations, it is not the proper solution in all circumstances. The **Cynefin** framework (Snowden and Boone 2007) is a sense-making framework that helps us understand the situation in which we have to operate and decide on a situation-appropriate approach. It defines and compares the characteristics of five different domains: **simple**, **complicated**, **chaotic**, **complex**, and a fifth domain, **disorder**, which occurs when you don't

know which other domain you are in (see Figure 1.3). I will use the Cynefin framework to discuss situations in which Scrum is and is not a good fit.

First, it is important to realize that the many facets of software development and support will not fit nicely into just one Cynefin domain. Software development is a rich endeavor, with aspects that overlap and activities that fall into all of the different domains (Pelrine 2011). So, while most software development work falls in the domains of complicated or complex, to boldly claim that software development is a complex domain would be naive, especially if we define software development to include the spectrum of work ranging from innovative new-product development, ongoing software product maintenance, and operations and support.

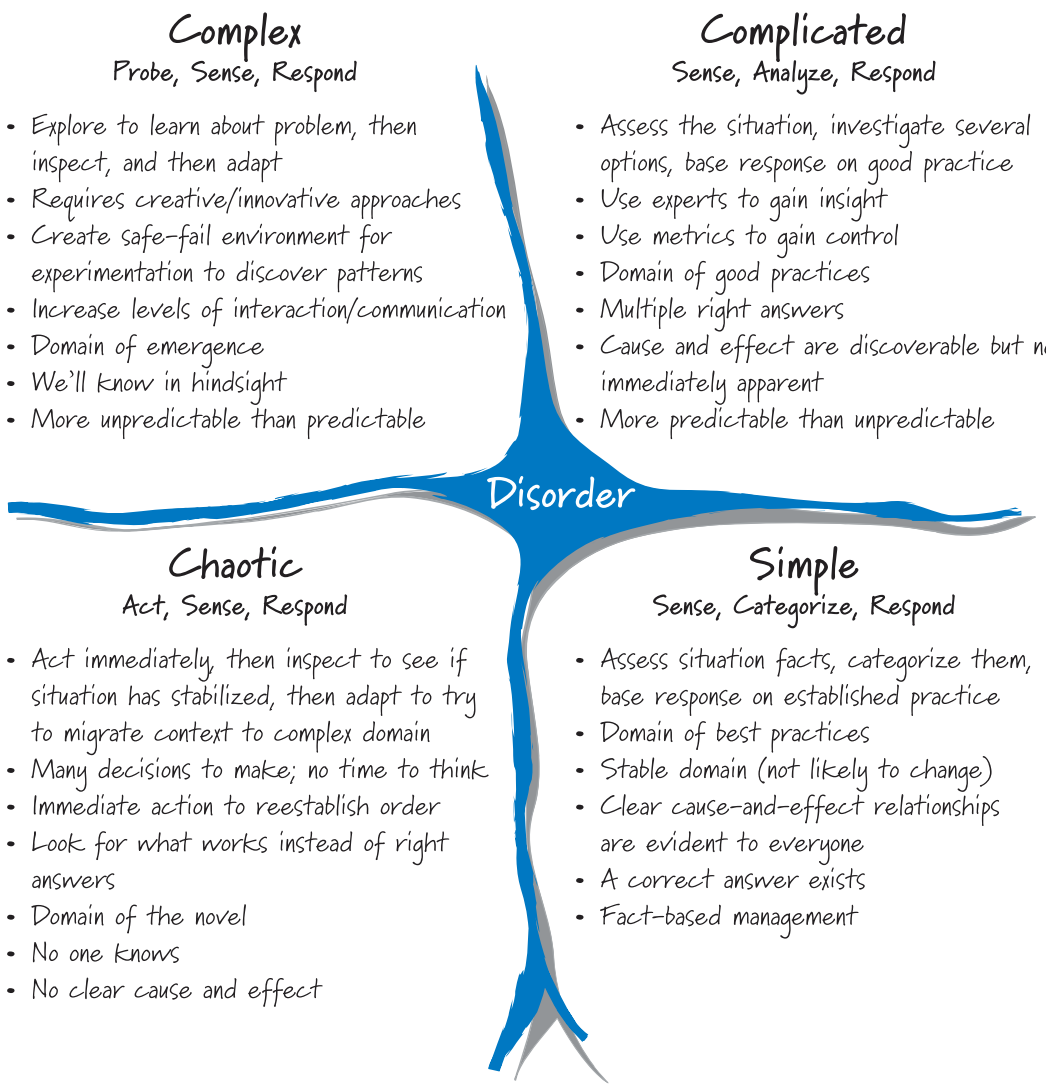


FIGURE 1.3 Cynefin framework

Complex Domain

When dealing with complex problems, things are more unpredictable than they are predictable. If there is a right answer, we will know it only with hindsight. This is the domain of **emergence**. We need to explore to learn about the problem, then **inspect and adapt** based on our learning. Working in complex domains requires creative and innovative approaches. Routine, cookie-cutter solutions simply don't apply. We need to create a safe-fail environment for experimentation so that we can discover important information. In this environment high levels of interaction and communication are essential. Innovative new-product development falls into this category as does enhancing existing products with innovative new features.

Scrum is particularly well suited for operating in a complex domain. In such situations our ability to probe (explore), sense (inspect), and respond (adapt) is critical.

Complicated Domain

Complicated problems are the domain of good practices dominated by experts. There might be multiple right answers, but expert diagnosis is required to figure them out. Although Scrum can certainly work with these problems, it might not be the best solution. For example, a performance optimization effort that calls for adjusting parameters to find the best overall system performance would be better served by assembling experts and letting them assess the situation, investigate several options, and base their response on good practice. Much of day-to-day software maintenance (dealing with a flow of product support or defect issues) falls into this category. This is also where many of the tactical, quantitative approaches like Six Sigma are particularly well suited, although these tactical approaches can also apply with simple domains.

Simple Domain

When dealing with simple problems, everyone can see cause and effect. Often the right answer is obvious and undisputed. This is the domain of legitimate best practices. There are known solutions. Once we assess the facts of our situation, we can determine the proper predefined solution to use. Scrum can be used for simple problems, but it may not be the most efficient tool for this type of problem. Using a process with a well-defined, repeatable set of steps that are known to solve the problem would be a better fit. For example, if we want to reproduce the same product over and over again, a well-defined assembly-line process would be a better fit than Scrum. Or deploying the same commercial-off-the-shelf (COTS) product into the 100th customer environment might best be completed by repeating a well-defined and proven set of steps for installing and configuring the product.

Chaotic Domain

Chaotic problems require a rapid response. We are in a crisis and need to act immediately to prevent further harm and reestablish at least some order. For example, suppose a university published an article stating that our product has a flawed algorithm that is producing erroneous results. Our customers have made substantial business investments based on the results from our product, and they are filing lawsuits against us for large damages. Our lead algorithm designer is on holiday in the jungles of Borneo and can't be reached for two more weeks. Scrum is not the best solution here. We are not interested in prioritizing a backlog of work and determining what work to perform in the next iteration. We need the ability to act immediately and decisively to stem the bleeding. With chaotic problems, someone needs to take charge of the situation and act.

Disorder

You are in the disorder domain when you don't know which of the other domains you are in. This is a dangerous place to be because you don't know how to make sense of your situation. In such cases, people tend to interpret and act according to their personal preference for action. In software development, many people are familiar with and therefore have a personal preference for phase-based, sequential approaches that work well in simple domains. Unfortunately, as I will discuss in Chapter 3, these tend to be a rather poor fit for much of software development. When you are in the disorder domain, the way out is to break down the situation into constituent parts and assign each to one of the other four domains. You are not trying to apply Scrum in the disorder domain; you are trying to get out of this domain.

Interrupt-Driven Work

Scrum is not well suited to highly interrupt-driven work. Say you run a customer support organization and you want to use Scrum to organize and manage your support activities. Your product backlog is populated on a continuous basis as you receive support requests via phone or email. At no point in time do you have a product backlog that extends very far into the future, and the content and order of your backlog could change frequently (perhaps hourly or every few minutes).

In this situation, you will not be able to reliably plan iterations of a week or more because you won't know what the work will be that far into the future. And, even if you think you know the work, there is a very good likelihood that a high-priority support request will arrive and preempt any such forward-looking plans.

In interrupt-driven environments you would be better off considering an alternative agile approach called **Kanban**. Kanban is not a stand-alone process solution, but instead an approach that is overlaid on an existing process. In particular, Kanban advocates that you

- Visualize how the work flows through the system (for example, the steps that the support organization takes to resolve a support request)
- Limit the work in process (WIP) at each step to ensure that you are not doing more work than you have the capacity to do
- Measure and optimize the flow of the work through the system to make continuous improvements

The sweet spots for Kanban are the software maintenance and support areas. Some Kanban practitioners point out that Kanban's focus on eliminating overburden (by aligning WIP with capacity) and reducing variability in flow while encouraging an evolutionary approach to change makes it appropriate to use in complex domains as well.

Scrum and Kanban are both agile approaches to development, and each has strengths and weaknesses that should be considered once you make sense of the domain in which you are operating. In some organizations both Scrum and Kanban can be used to address the different system needs that coexist. For example, Scrum can be used for new-product development and Kanban for interrupt-driven support and maintenance.

Closing

Scrum is not a silver bullet or a magic cure. Scrum can, however, enable you to embrace the changes that accompany all complex **product development efforts**. And it can, and has, worked for Genomica and many other companies that decided to employ an approach to software development that better matched their circumstances.

Although the Scrum framework is simple, it would be a mistake to assume that Scrum is easy and painless to apply. Scrum doesn't prescriptively answer your process questions; instead, it empowers teams to ask and answer their own great questions. Scrum doesn't give individuals a cookbook solution to all of their organizational maladies; instead, Scrum makes visible the dysfunctions and waste that prevent organizations from reaching their true potential.

These realizations can be painful for many organizations. However, if they move past the initial discomfort and work to solve the problems Scrum unearths, organizations can take great strides in terms of both their software development process and products and their levels of employee and customer satisfaction.

The rest of the book is devoted to discussing the essential aspects of Scrum. I will begin with a description of the entire Scrum framework, including its roles, activities, artifacts, and **rules**. Who knows; if you use Scrum in the right way and in the proper conditions, perhaps you too will deliver value as successfully as my wife did on that fateful day back in 2000.

Part I

Core Concepts

This page intentionally left blank

Chapter 2
Scrum Framework

This chapter provides an overview of the Scrum framework with a primary focus on its practices, including roles, activities, and artifacts. Subsequent chapters will provide a deeper treatment of each of these practices, including an in-depth look at the principles that underlie the practices.

Overview

Scrum is not a standardized process where you methodically follow a series of sequential steps that are guaranteed to produce, on time and on budget, a high-quality product that delights customers. Instead, Scrum is a **framework** for organizing and managing work. The Scrum framework is based on a set of values, principles, and practices that provide the foundation to which your organization will add its unique implementation of relevant engineering practices and your specific approaches for realizing the Scrum practices. The result will be a version of Scrum that is uniquely yours.

To better grasp the framework concept, imagine that the Scrum framework is like the foundation and walls of a building. The Scrum values, principles, and practices would be the key structural components. You can't ignore or fundamentally change a value, principle, or practice without risking collapse. What you can do, however, is customize inside the structure of Scrum, adding fixtures and features until you have a process that works for you.

Scrum is a refreshingly simple, people-centric framework based on the values of honesty, openness, courage, respect, focus, trust, empowerment, and collaboration. Chapter 3 will describe the Scrum principles in depth; subsequent chapters will highlight how specific practices and approaches are rooted in these principles and values.

The Scrum practices themselves are embodied in specific roles, activities, artifacts, and their associated rules (see Figure 2.1).

The remainder of this chapter will focus on Scrum practices.

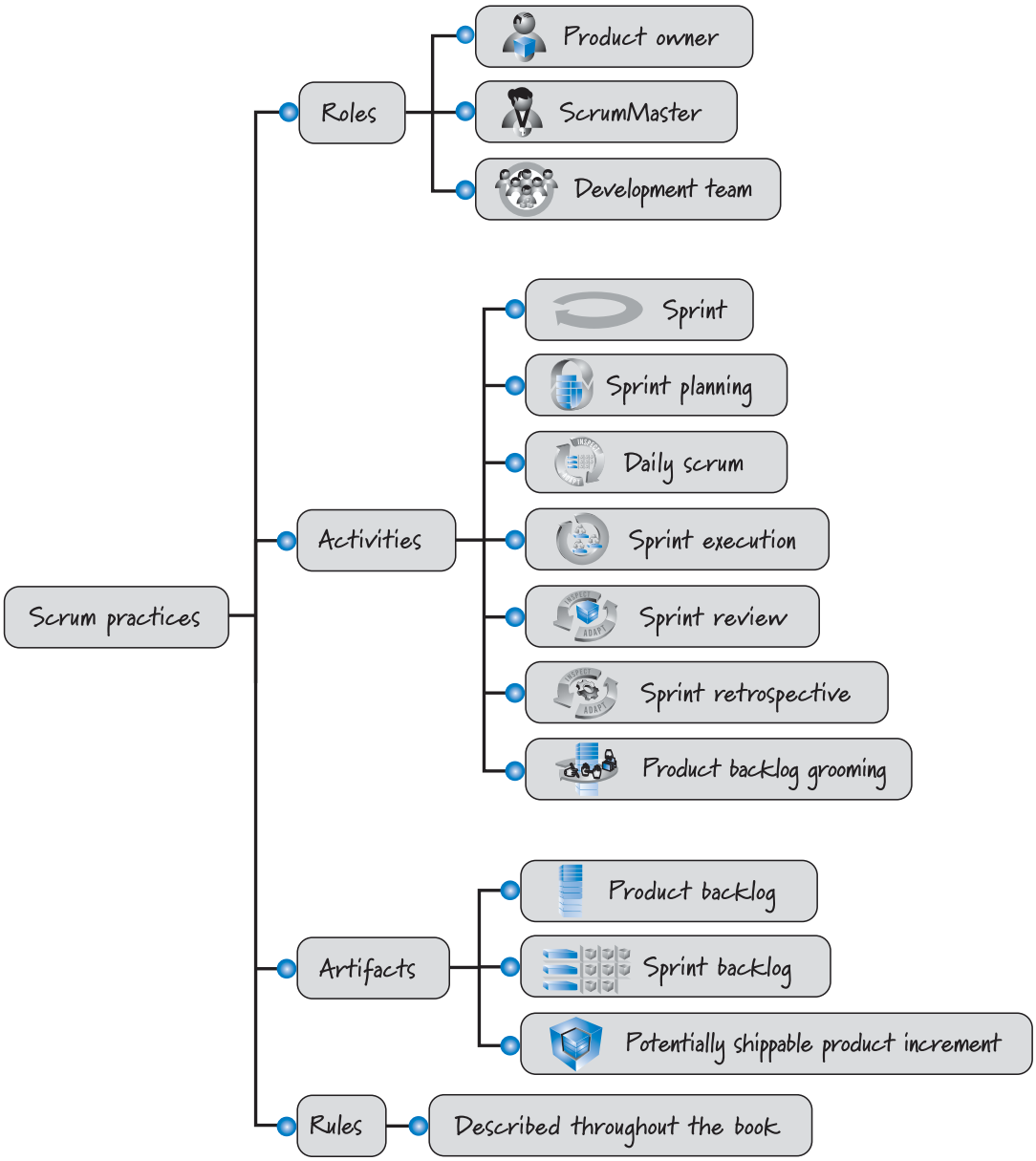


FIGURE 2.1 Scrum practices

Scrum Roles

Scrum development efforts consist of one or more **Scrum teams**, each made up of three Scrum roles: **product owner**, **ScrumMaster**, and the **development team** (see Figure 2.2). There can be other roles when using Scrum, but the Scrum framework requires only the three listed here.

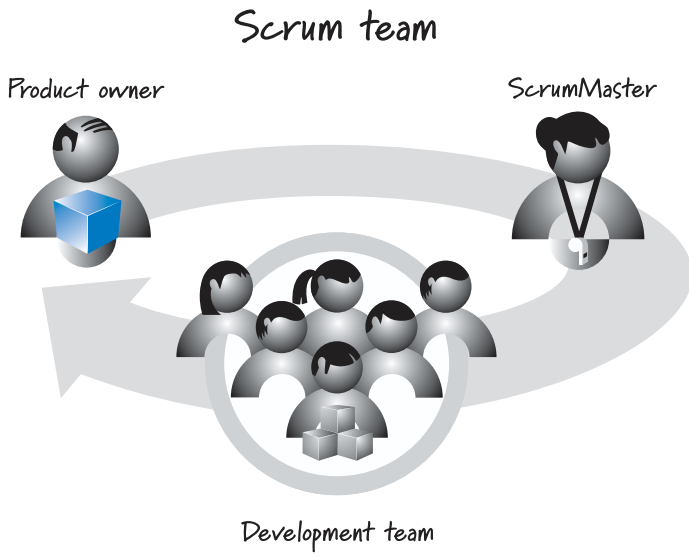


FIGURE 2.2 Scrum roles

The product owner is responsible for what will be developed and in what order. The ScrumMaster is responsible for guiding the team in creating and following its own process based on the broader Scrum framework. The development team is responsible for determining how to deliver what the product owner has asked for.

If you are a manager, don't be concerned that "manager" doesn't appear as a role in Figure 2.2; managers still have an important role in organizations that use Scrum (see Chapter 13). The Scrum framework defines just the roles that are specific to Scrum, not all of the roles that can and should exist within an organization that uses Scrum.

Product Owner

The product owner is the empowered central point of product leadership. He[1] is the single authority responsible for deciding which features and functionality to build and the order in which to build them. The product owner maintains and communicates to all other participants a clear vision of what the Scrum team is trying to achieve. As such, the product owner is responsible for the overall success of the solution being developed or maintained.

It doesn't matter if the focus is on an external product or an internal application; the product owner still has the obligation to make sure that the most valuable work possible, which can include technically focused work, is always performed. To

1. In this book the product owner will always be referred to as "he" or "him" and the ScrumMaster as "she" or "her." This is consistent with the visual representation of each role within the figures.